Lab 9 — Pre-Lab Reading Guide

Walk-through of the Sensor Simulation Notebook

EE0849 Introduction to Robotics

Table of contents

1 How to use this guide

This document mirrors the Lab 9 Jupyter notebook cell by cell. Read it **before** lab so that during the session you can focus on typing, plotting, and debugging rather than reading. Every major cell of the notebook has three things below:

1. **What the cell does** — the purpose in one or two sentences.

2. **Key code** — the exact snippet you will see.
3. **What to notice** — the one idea worth remembering from that cell.

The notebook is split into four parts plus a wrap-up:

- **Part 1** simulates a 2D LiDAR.
- **Part 2** fuses one scan into a log-odds occupancy grid.
- **Part 3** drives the robot to eight poses and fuses all scans.
- **Part 4** picks out a red cone in a synthetic camera image with OpenCV.

Time budget in lab: **30 min per part**. If you have done the pre-lab reading you will comfortably finish Parts 1–3 in class.

---

2 Pre-lab questions (answer on paper before lab)

Q1. You build a scan with 360 beams evenly spaced over $[0, 2\pi)$, so the angular step is $\Delta\alpha = 1^\circ$. Beam index $i = 90$ returns $r = 5$ m. What are the (x, y) coordinates of the hit **in the sensor frame**?

Q2. A cell has log-odds $\ell = 0$. Three beams pass through it and one beam ends on it. With $\ell_{\text{free}} = -0.4$ and $\ell_{\text{occ}} = +0.85$, what is the final log-odds? What probability does that correspond to? Is the cell "free" or "occupied"?

Q3. Why do we threshold a red cone in **HSV** space instead of directly in **BGR**?

Answers are at the bottom of the notebook (Instructor Pre-lab answers) — check yourself only after attempting.

---

3 Part 1 — Simulating a 2D LiDAR

The first block imports libraries and seeds the random generator. `numpy` for math, `matplotlib` for plots, `cv2` for images (Part 4), `skimage.draw.line` for Bresenham rasterization (Part 2):

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
from skimage.draw import line
rng = np.random.default_rng(0)
```

Notice: a fixed random seed (`default_rng(0)`) means noise experiments in Part 3 produce the same figure every run. You can change `0` to any integer to shuffle.

3.1 `make_world()` — build the environment

The environment is just a **list of line segments**. Four segments form the outer 10 m × 10 m room; six more form an L-shaped obstacle in the middle.

```
walls = [
    ((0.0, 0.0), (10.0, 0.0)),
    ((10.0, 0.0), (10.0, 10.0)),
    ((10.0, 10.0), (0.0, 10.0)),
    ((0.0, 10.0), (0.0, 0.0)),
]
obstacle = [((4,4),(6,4)), ((6,4),(6,6)), ((6,6),(5,6)),
```

```
            ((5,6),(5,5)), ((5,5),(4,5)), ((4,5),(4,4))]
return walls + obstacle
```

Notice: representing the world as segments lets ray-casting be a pure math problem — no image, no rasterization yet. You can add or move obstacles by editing this list.

3.2 `plot_world()` and the first figure

A helper that draws all segments in black and (optionally) the robot pose as a red dot with a heading arrow. The first figure shows the empty room with the robot at pose `(2, 2, 0)`, i.e. in the bottom-left facing +x.

Notice: `ax.set_aspect('equal')` is the detail that keeps the map from looking stretched. Always set it when plotting 2D geometry.

3.3 `ray_segment_intersection(origin, direction, seg)`

Purpose: return the distance $t \geq 0$ along a ray until it crosses a given segment, or `np.inf` if it never does.

The math: we solve $O + t\,D = A + u\,(B - A)$ for (t, u) and accept the hit only if $t \geq 0$ (in front of the robot) and $0 \leq u \leq 1$ (on the segment, not on its extension).

```
def ray_segment_intersection(origin, direction, seg):
    Ox, Oy = origin
    Dx, Dy = direction
    (Ax, Ay), (Bx, By) = seg
    Ex, Ey = Bx - Ax, By - Ay
    det = Dy * Ex - Dx * Ey
    if abs(det) < 1e-12:
        return np.inf      # parallel
    rx, ry = Ax - Ox, Ay - Oy
    t = (ry * Ex - rx * Ey) / det
    u = (Dx * ry - Dy * rx) / det
    if t >= 0.0 and 0.0 <= u <= 1.0:
        return t
    return np.inf
```

Sanity check in the cell: a ray from $(2, 2)$ pointing in $+x$ should hit the east wall at $x = 10$, so $t = 8.00$ m. If your function prints anything else, you have a sign or index bug.

Notice: `abs(det) < 1e-12` catches the **parallel case** — when the ray and the segment have the same direction, the linear system has no unique solution and `det = 0` would divide by zero. We just say "no hit" and move on.

3.4 `cast_ray(origin, angle, segments, max_range=10)`

Purpose: loop `ray_segment_intersection` over all segments, return the **smallest** positive distance, clipped at `max_range`. One beam, one number.

```
def cast_ray(origin, angle, segments, max_range=10.0):
    direction = (np.cos(angle), np.sin(angle))
    best = max_range
    for seg in segments:
        t = ray_segment_intersection(origin, direction, seg)
        if t < best:
            best = t
    return best
```

Quick check from $(2, 2)$: four bearings (0°, 45°, 90°, 180°) should return (8 m, 2.83 m, 8 m, 2 m). The 45° beam bumps into the L-shape's corner at $\sqrt{2^2 + 2^2} \approx 2.83$ m.

Notice: `max_range` is the sensor's physical limit. A beam that sees no obstacle within `max_range` returns `max_range` itself — **this is not a hit**. Part 2 treats it differently than a real hit.

3.5 `scan(pose, segments, n_beams=360)`

Purpose: sweep 360 beams around the robot, return parallel arrays of bearings α and ranges r.

```
def scan(pose, segments, n_beams=360, max_range=10.0):
    xR, yR, theta = pose
    alphas = np.linspace(0.0, 2*np.pi, n_beams, endpoint=False)
    ranges = np.zeros(n_beams)
    for i, a in enumerate(alphas):
        ranges[i] = cast_ray((xR, yR), theta + a, segments, max_range)
    return alphas, ranges
```

The single most important line: `theta + a`. The bearing α is in the **sensor frame** (what the LiDAR chip reports), but the ray lives in the **world frame**. Adding the robot's heading θ rotates each beam into world coordinates. Forget this and your scan only looks right when $\theta = 0$.

The cell then plots the scan: a gray fan of 360 rays emanating from $(2, 2)$ with blue dots at the endpoints. You will see a dark wedge behind the L-shape — the **shadow**.

---

4 Part 2 — One Scan to an Occupancy Grid

4.1 Grid parameters

```
grid_origin = (0.0, 0.0)     # world coords of cell (0, 0)
res         = 0.1            # metres per cell
M = N = int(10 / res)        # 100 x 100 cells
```

Notice: the resolution 0.1 m = 10 cm. Finer cells give sharper edges but multiply memory and scan-update cost (both by $1/\text{res}^2$).

4.2 `world_to_grid(x, y, origin, res)` and `grid_to_world(i, j, origin, res)`

Convert between continuous world coordinates (metres) and integer cell indices. `world_to_grid` uses Python `int()`, which truncates toward zero — fine because our world has $x \geq 0$, $y \geq 0$. `grid_to_world` returns the **cell center** by adding 0.5.

```
def world_to_grid(x, y, origin, res):
    i = int((x - origin[0]) / res)
    j = int((y - origin[1]) / res)
    return i, j

def grid_to_world(i, j, origin, res):
    x = origin[0] + (i + 0.5) * res
    y = origin[1] + (j + 0.5) * res
    return x, y
```

The sanity checks round-trip three points — e.g. $(2.55, 3.15) \to (25, 31) \to (2.55, 3.15)$. If yours round-trip to a different cell, check whether `res` is being treated as an integer somewhere.

Notice: we store the grid as `grid[i, j]` with `i` = x-index. Be consistent everywhere. The wrap-up lists "passing `(j, i)` instead of `(i, j)` to `line()`" as a top-three pitfall.

4.3 `update_grid(...)` — the heart of the lab

Purpose: take one scan and fold it into the log-odds grid using the **inverse sensor model**.

For each beam:

1. Compute endpoint in world coords: $e = (x_R + r\cos(\theta+\alpha),\ y_R + r\sin(\theta+\alpha))$.
2. Convert robot cell (r_i, r_j) and endpoint cell (e_i, e_j) to grid indices.
3. Use `skimage.draw.line(ri, rj, ei, ej)` to get all cells along the ray (Bresenham).
4. Add $\ell_{\text{free}} = -0.4$ to every cell **except the last**.
5. Add $\ell_{\text{occ}} = +0.85$ to the endpoint cell, **only if the ray actually hit something**.
6. Clamp the whole grid to $[-5, +5]$ so a long-observed cell can still be revised.

```
ii, jj = line(ri, rj, ei, ej)
if len(ii) > 1:
    grid[ii[:-1], jj[:-1]] += l_free
if r < max_range - 1e-6:
    grid[ii[-1], jj[-1]] += l_occ
else:
    grid[ii[-1], jj[-1]] += l_free    # max-range ray: treat as another 'free' step
```

Once the grid is filled, convert to probability with the sigmoid: $p = 1/(1+e^{-\ell})$. The cell then plots two heat maps side by side — the raw log-odds (red/blue) and the probability map (grayscale).

Notice: the -5 to $+5$ clamp keeps the map **correctable**. Without it, a cell seen as free 100 times would be at $\ell = -40$ and a single contrary observation ($+0.85$) could not flip it. Clamping is cheap insurance against dynamic environments and outlier measurements.

4.4 Step 2.4 — the occlusion wedge

Behind the L-shape, no ray ever reaches — those cells stay at $\ell = 0$, which is $p = 0.5$ (mid-gray). **Occluded, not empty**. The robot has to move to see them. This is the motivation for Part 3.

---

5 Part 3 — Multi-scan Mapping

5.1 Eight poses around the room

```
poses = [
    # 4 corners (face toward room center)
    (2.0, 2.0, +np.pi/4),
    (2.0, 8.0, -np.pi/4),
    (8.0, 8.0, -3*np.pi/4),
    (8.0, 2.0, +3*np.pi/4),
    # 4 mid-edges (face inward)
    (5.0, 2.0, +np.pi/2),
    (2.0, 5.0,  0.0),
    (5.0, 8.0, -np.pi/2),
    (8.0, 5.0, +np.pi),
]
```

The robot teleports to four room corners plus four mid-edge points — eight viewpoints total. The loop fuses one scan per pose into the **same** log-odds grid, so contributions from different viewpoints add linearly in log-odds space (that is equivalent to multiplying probabilities, i.e. standard Bayesian fusion).

The plotted progression is a 2×4 grid of panels: after scan 1 (only one wedge observed), after scan 2 (two wedges), etc. By scan 8 essentially every cell in the room has been visited by several beams. Why 8 and not 4? With only 4 corner poses the hysteresis ("strictly free" only below $p = 0.2$) leaves many lightly-visited cells in the *unknown* class. 8 poses give enough overlap that the simple and hysteresis maps agree almost everywhere.

At the end:

```
Log-odds range after 8 scans: [-5.00, +5.00]     # clamp kicked in
Fraction of cells still unknown (|l| < 0.1): very small
```

Notice: this is **mapping with known poses**, not SLAM. We *assumed* the eight pose triples are exact. In a real robot, each pose comes from odometry + some pose filter; errors there would smear the map. You will see SLAM in a later course.

5.2 Noise experiment — `sigma = 0.05 m`

We repeat the eight-scan fuse, but this time we perturb each range measurement with Gaussian noise of standard deviation 5 cm — roughly one cell. Side-by-side plot shows clean vs noisy maps.

Where does the noise hurt most? **The wall edges**. A ray that measures slightly short stamps a wrong cell just inside the wall as occupied; a ray that measures slightly long puts the occupied mark one cell past the wall. Interior free cells average out — they are visited by dozens of beams and random noise cancels.

Notice: that is the entire reason we fuse many scans instead of trusting one. Independent noise on many observations averages out in log-odds space.

5.3 Step 3.4 — resolution trade-off

If we halved the cell size to 5 cm:

- **(a) Memory**: $\times 4$ (area scales like $1/\text{res}^2$).
- **(b) Sharpness**: walls localized to one 5 cm cell, not one 10 cm cell — better.
- **(c) More rays needed?** Yes. At 10 m range with 1° spacing, adjacent beams are ~ 17 cm apart, so they skip over several 5 cm cells at long range. To cover every cell you would need ~ 1080 beams (every $1/3^\circ$).

This is the classical sensing–compute–memory triangle: you cannot get sharper maps for free.

5.4 Step 3.5 — From probability to a binary occupancy grid

Everything up to here produced a **probability** map. A planner like Dijkstra or A* wants a **binary** map: every cell is either *free* or *blocked*. This step collapses one into the other. Two common choices:

- **Simple threshold**: `occ_simple = (prob > 0.5)`. One line. Every cell with $p > 0.5$ is blocked, everything else is free.
- **Hysteresis** (safer): three classes.
  - $p > 0.65 \rightarrow$ **occupied**
  - $p < 0.45 \rightarrow$ **free**
  - $0.45 \leq p \leq 0.65 \rightarrow$ **unknown** (never observed)

  A safety-minded planner then treats `blocked = (occupied OR unknown)`. The robot refuses to drive through cells it has never seen.

```
prob = 1.0 / (1.0 + np.exp(-grid_log))
```

```
# (a) Simple threshold
occ_simple = (prob > 0.5).astype(np.uint8)

# (b) Hysteresis
occ_hyst = np.full_like(prob, 1, dtype=np.uint8)  # default: unknown
occ_hyst[prob < 0.45] = 0                         # free
occ_hyst[prob > 0.65] = 2                         # occupied
planner_blocked = (occ_hyst != 0).astype(np.uint8)
```

Why the free bound is 0.45 and not, say, 0.2: a single ray passing through a cell drops its log-odds by $\ell_{\text{free}} = -0.4$, giving $p \approx 0.40$. A strict bound like 0.2 would need **four** passes before a cell is "free enough", which with only eight scans leaves many edge cells stuck in the unknown class. 0.45 is the loosest bound that still separates "we saw this" from "we never saw this".

The notebook plots all three side by side: the smooth probability map, the crisp `p > 0.5` threshold, and the conservative hysteresis mask.

Why not just use `p > 0.5` everywhere? An unobserved cell has $\ell = 0 \Rightarrow p = 0.5$ exactly. The simple threshold classifies it as *free* (just barely under), so the planner will happily route the robot through occluded space it has never sensed. That is how a real-world robot ends up driving into a wall "because the planner said it was clear."

Trade-off: the hysteresis mask blocks more cells, so some genuinely free but unexplored corridors may be unreachable until the robot moves to observe them. That is the right default — a missed corridor is cheap, a collision is not.

Notice: this `planner_blocked` map is **the** hand-off to Week 11. The Dijkstra / A* search operates on exactly this binary grid.

---

6 Part 4 — A Taste of Vision with OpenCV

6.1 `synthetic_scene()` — build a fake camera image

A 640 × 480 BGR image with:

- a red cone on the left (triangle),
- a yellow ball in the center,
- a blue box on the right,
- a gray floor and a dim bluish "sky".

OpenCV uses **BGR** channel order, not RGB — a frequent gotcha. Matplotlib wants RGB, so we convert with `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` before plotting.

Notice: a camera answers "*what* is there?"; it cannot tell you "how far." LiDAR is the other way round. Part 4 is the first step toward fusing the two.

6.2 Step 4.2 — HSV threshold for red

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
lower_red = np.array([0, 120, 70])
upper_red = np.array([10, 255, 255])
mask = cv2.inRange(hsv, lower_red, upper_red)
```

Why HSV and not BGR? HSV separates **H**ue (color) from **V**alue (brightness). Under shadow or sunlight the cone's hue stays roughly constant, but every BGR channel changes. A fixed BGR threshold would lose

the cone whenever the lighting changed; an HSV hue threshold is robust.

The cell plots three panels — original, HSV rendered as RGB (intentionally weird-looking), and the binary mask — and prints the number of red pixels.

Notice: true red spans two hue ranges, $[0, 10]$ and $[170, 180]$, because hue wraps at 180 in OpenCV's 8-bit HSV. For the synthetic cone one range is enough; for a real photo you would combine both masks with `cv2.bitwise_or`.

6.3 Step 4.3 — Contour, bounding box, centroid

```
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cnt = max(contours, key=cv2.contourArea)
x, y, w, h = cv2.boundingRect(cnt)
cx, cy = x + w // 2, y + h // 2
```

- `findContours` returns all connected regions in the mask.
- `max(..., key=cv2.contourArea)` keeps only the largest — robust against tiny noise blobs.
- `boundingRect` gives the axis-aligned box; its midpoint is our **target pixel** (c_x, c_y).

The visualization draws a green box and a yellow centroid dot, plus the contour area in px^2.

Notice: `RETR_EXTERNAL` ignores holes — for a solid red cone this is what we want. If you ever track a doughnut-shaped marker, switch to `RETR_LIST`.

6.4 Step 4.4 — Pixel to steering command

```
img_center = img.shape[1] // 2    # 320
band = 50                         # pixel dead zone
if   cx < img_center - band: action = 'TURN LEFT'
elif cx > img_center + band: action = 'TURN RIGHT'
else:                       action = 'GO STRAIGHT'
```

With the cone at $c_x = 150$ and the image center at 320, the centroid is well to the left of the dead zone, so the command is **TURN LEFT**. Move the cone's coordinates in `synthetic_scene` to test the other cases.

Notice: the 50-pixel dead zone prevents jitter. Without it, tiny pixel noise would flip between LEFT and RIGHT every frame even when the cone is essentially dead-ahead.

6.5 Step 4.5 — LiDAR vs camera, one line each

- LiDAR: "where" — metric geometry, no appearance.
- Camera: "what" — appearance, no depth.
- Real robots fuse both.

---

7 Wrap-up — things to remember

Time budget: 30 / 30 / 30 / 30 minutes per part. Parts 1–3 are the core; Part 4 is a taste of vision that you can finish at home if needed.

Top pitfalls (instructor has seen each of these cost a student 20 minutes):

1. **Forgetting `theta + alpha`** — the map only looks right when the robot is facing east.
2. **Not clamping the log-odds** — cells saturate and never revise.

3. **Marking the endpoint as occupied even when `r == max_range`** — false obstacles appear in open space.
4. **Passing `(j, i)` instead of `(i, j)` to `line()`** — the map is transposed.

What this produces for next week: the binary `planner_blocked` mask from Step 3.5 is exactly what Dijkstra and A* search. You are building the input for next week.

---

8 Pre-lab answers (do not read until you have tried)

- **Q1.** With $\Delta\alpha = 1^\circ$ and beam 90, $\alpha = 90^\circ = \pi/2$. So $x^S = r\cos(\pi/2) = 0$, $y^S = r\sin(\pi/2) = 5$. Sensor-frame coordinates: $(0, 5)$ m.
- **Q2.** $\ell = 0 + 3(-0.4) + 1(+0.85) = -0.35$. Then $p = 1/(1 + e^{+0.35}) = 0.413$. Slightly **leaning free** (under 0.5).
- **Q3.** HSV separates hue from brightness, so the cone's hue is roughly constant under shadow or sunlight. In BGR, every channel changes with lighting and a fixed threshold fails.